Mantık Devreleri - Ödev 2

Deney yaprakları 1-4

Efe Özan - 170425062

29.10.2026

Deneylerin hesapları ve ara kayıtları burada. Her deneyin programı ve doğrulama kodu `kodlar/` klasöründedir: `ikili_toplayici.py`, `ikili_cikarici.py`, `ikili_carpici.py`, `ikili_bolucu.py` ve `dogrulama_1..4.py`.

Deney 1 - Elde adımlarını gösteren ikili toplayıcı

1. Bir sütunda iki bit (a_i, b_i) ve gelen elde (c_{in}) toplandığında sonuç biti $s_i = a_i \oplus b_i \oplus c_{\text{in}}$, çıkan elde c_{out} ise en az iki giriş 1 olduğunda 1 olur. Sağdan sola çalışan algoritmayı `ikili_toplayici.py` içinde kodladım.

a_i	b_i	Gelen elde (c_{in})	Sonuç biti (s_i)	Çıkan elde (c_{out})
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

2. Dört işlemi programdan önce hesapladım: `0101 + 0011` ($c_0 = 0$) $\rightarrow$ `1000` elde `0` (8); `1111 + 0001` ($c_0 = 0$) $\rightarrow$ `0000` elde `1` (16); `1111 + 0000` ($c_0 = 1$) $\rightarrow$ `0000` elde `1` (16); `1111 + 1111` ($c_0 = 1$) $\rightarrow$ `1111` elde `1` (31). Programın ürettiği sütun kayıtları (`a_i,b_i,c_in -> s_i,c_out`):

İşlem	Sütun 0 (1)	Sütun 1 (2)	Sütun 2 (4)	Sütun 3 (8)	Toplam	Son elde	Onlu
`0101+0011` ($c_0 = 0$)	`1,1,0->0,1`	`0,1,1->0,1`	`1,0,1->0,1`	`0,0,1->1,0`	`1000`	`0`	8
`1111+0001` ($c_0 = 0$)	`1,1,0->0,1`	`1,0,1->0,1`	`1,0,1->0,1`	`1,0,1->0,1`	`0000`	`1`	16
`1111+0000` ($c_0 = 1$)	`1,0,1->0,1`	`1,0,1->0,1`	`1,0,1->0,1`	`1,0,1->0,1`	`0000`	`1`	16
`1111+1111` ($c_0 = 1$)	`1,1,1->1,1`	`1,1,1->1,1`	`1,1,1->1,1`	`1,1,1->1,1`	`1111`	`1`	31

3. Dört bitlik bütün $16 \times 16 \times 2 = 512$ testi denedim. Hepsinde 4 bitlik çıktının değeri ile son_elde $\times 16$ toplamı referans onlu toplamla birebir aynı çıktı (**512/512 doğru**; 256 testte son elde 0, 256 testte 1).

4. Boş girdi (`""`), `0/1` dışındaki karakter (`"0121"`), farklı uzunluk (`"0101"` ve `"11"`) ve geçersiz başlangıç eldesi hata veriyor. n bitlik toplamada son elde n. basamağa karşılık geldiği için ağırlığı **2^n**'dir; yok saymak toplamdan **2^n** kaybettirir. Sınır örneği: $n = 4$ için `1111 + 0001` işleminde son elde (`1`) atılırsa 16 yerine `0000` (0) kalır ve $2^4 = 16$ kaybedilir.

YZ önerisi değerlendirmem (Deney 1): YZ'nin verdiği tam toplayıcı tablosunu ve sağdan sola döngü yapısını 4 zorunlu işlemde kağıt üzerinde adım adım takip ederek doğruladım.

Deney 2 - Borç alma ve 2 tümleyeniyle çıkarma

1. `ikili_cikarici.py` içinde birbirini çağırmayan iki ayrı fonksiyon yazdım:

- **Borç alarak çıkarma (`borc_alarak_cikar`):** Sağdan sola tam çıkarıcı tablosuyla ilerler. Son borç `0` ise fark pozitiftir (`0000` ise sıfırdır). Son borç `1` ise $M < N$ olduğu için büyüklük $N - M$ borç zinciriyle bulunur ve başına `-` konur.
- **2 tümleyeniyle çıkarma (`iki_tumleyeniyle_cikar`):** N'nin bitlerini tersleyip ($\sim N$) başlangıç eldesi $c_0 = 1$ ile $M + \sim N + 1$ toplamını yapar. Toplam `0000` ise sıfır olarak ayrılır. Son elde `1` ise sonuç pozitif (`ham_toplam`), son elde `0` ise sonuç negatiftir ve `ham_toplam`'ın 2 tümleyeni alınarak `-büyüklük` yazılır.

2. Beş zorunlu işlemin iki yöntemle çözümü ve ara kayıtları (borç ve elde zincirleri sağdan sola, yani sütun $0 \to 3$ sırasıyla verilmiştir):

İşlem	Ham fark	Borç (sütun 0→3)	S.Borç	$\sim N$	4b $(-N)$	Ham top.	Elde (sütun 0→3)	S.Elde	Sonuç
`1110 - 0011`	`1011`	`1,1,0,0`	`0`	`1100`	`1101`	`1011`	`0,0,1,1`	`1`	`1011` (11)
`0011 - 1110`	`0101`	`0,0,1,1`	`1`	`0001`	`0010`	`0101`	`1,1,0,0`	`0`	`-1011` (-11)
`1000 - 0001`	`0111`	`1,1,1,0`	`0`	`1110`	`1111`	`0111`	`0,0,0,1`	`1`	`0111` (7)
`0101 - 0101`	`0000`	`0,0,0,0`	`0`	`1010`	`1011`	`0000`	`1,1,1,1`	`1`	`0000` (0)
`0011 - 0000`	`0011`	`0,0,0,0`	`0`	`1111`	`0000`	`0011`	`1,1,1,1`	`1`	`0011` (3)

3. 0–15 aralığındaki **256 operand çiftinin** tamamında iki yöntem de onlu referans farkla aynı sonucu verdi (**256/256 doğru**).

4. Sıfırın (`0000`) 4-bit genişlikte 2 tümleyeni $\sim$ `0000` $+ 1 =$ `1111` $+ 1 = (1)$`0000` $\to$ `0000`'dır. Sabit 4-bit 2 tümleyeniyle $M +$ `0000` toplandığında `son_elde = 0` çıkar. En küçük karşı örnek **M = 0000**, **N = 0000** (sonuç 0) ve pozitif karşı örnek **M = 0001**, **N = 0000** (sonuç $+1$, negatif değil). Karar kuralı: $M + \sim N + 1$ kullanıldığında son_elde $= 1 \implies M \geq N$, son_elde $= 0 \implies M < N$ olur; 4-bit tümleyen kullanılırsa `son_elde == 1 veya N == 0000` kontrol edilmeli ve `0000` çıktısı işaretsiz sıfır yapılmalıdır.

YZ önerisi değerlendirmem (Deney 2): YZ'nin önerdiği iki yöntemi 256 testte onlu referansla karşılaştırarak doğruladım; $N =$ `0000` sınır durumunu elle hesapladım.

Deney 3 - Kısmi çarpımlarla ikili çarpıcı

1. Çarpanın sağdan $i.$ biti b_i için n bitlik kısmi çarpım ($b_i = 1$ ise A, $b_i = 0$ ise 0^n) sola i basamak kaydırılır; sağına i sıfır, soluna da toplam uzunluğu $n + m$ yapmak için $m - i$ sıfır eklenir. Kısmi çarpımları hazır çarpma kullanmadan bit düzeyi toplayıcıyla birleştirdim.

2. Üç zorunlu çarpım için $n + m$ bitlik hizalı kısmi çarpımlar ve onlu katkıları:

- **`1011 x 101`** ($11 \times 5 = 55$, $4 + 3 = 7$ bit): $i = 0, b_0 = 1 \to$ `0001011` (11); $i = 1, b_1 = 0 \to$ `0000000` (0); $i = 2, b_2 = 1 \to$ `0101100` (44) $\to$ Toplam: **`0110111`** ($11 + 0 + 44 =$ **55**).
- **`10111 x 110`** ($23 \times 6 = 138$, $5 + 3 = 8$ bit): $i = 0, b_0 = 0 \to$ `00000000` (0); $i = 1, b_1 = 1 \to$ `00101110` (46); $i = 2, b_2 = 1 \to$ `01011100` (92) $\to$ Toplam: **`10001010`** ($0 + 46 + 92 =$ **138**).
- **`11001 x 1010`** ($25 \times 10 = 250$, $5 + 4 = 9$ bit): $i = 0, b_0 = 0 \to$ `000000000` (0); $i = 1, b_1 = 1 \to$ `000110010` (50); $i = 2, b_2 = 0 \to$ `000000000` (0); $i = 3, b_3 = 1 \to$ `011001000` (200) $\to$ Toplam: **`011111010`** ($0 + 50 + 0 + 200 =$ **250**).

3. Sekiz test ve sıfır kısmi çarpımları atlayan 2. sürümle karşılaştırma (Sayım kuralı: başlangıç akümülatörü 0^{n+m} alınır; **Sürüm 1 her satır için 1 kez, toplamda m kez** toplayıcıyı çağırır; **Sürüm 2 yalnız $b_i = 1$ olan satırlar için 1 kez, toplamda birlerin sayısı kadar** çağırır):

#	Senaryo	İşlem	Çıktı ($n+m$ bit)	Onlu	Sürüm 1 Çağrı	Sürüm 2 Çağrı
1	Zorunlu 1 (11×5)	`1011 x 101`	`0110111`	55	3	2
2	Zorunlu 2 (23×6)	`10111 x 110`	`10001010`	138	3	2
3	Zorunlu 3 (25×10)	`11001 x 1010`	`011111010`	250	4	2
4	Çarpan sıfır (13×0)	`1101 x 000`	`0000000`	0	3	0
5	Çarpan bir (13×1)	`1101 x 001`	`0001101`	13	3	1
6	Çarpan 2'nin kuvveti (11×8)	`1011 x 1000`	`01011000`	88	4	1
7	Çarpan tüm bitleri 1 (15×7)	`1111 x 111`	`1101001`	105	3	3
8	Çarpılan 0, çarpan tüm 1 (0×15)	`0000 x 1111`	`00000000`	0	4	4

4. En büyük n ve m bitlik sayıların çarpımı $(2^n - 1)(2^m - 1) = 2^{n+m} - 2^n - 2^m + 1 < 2^{n+m} - 1$ olduğundan sonuç her zaman **n + m bit** içine sığar. $n + m - 1$ bitin yetmediğine karşı örnek: $n = 2, m = 2$ için `11 x 11 = 1001` ($3 \times 3 = 9$, 4 bit); $n + m - 1 = 3$ bit en çok `111` (7) tutabilir.

YZ önerisi değerlendirmem (Deney 3): YZ'nin önerdiği sol/sağ sıfır doldurma kuralını ($m - i$ sol sıfır, i sağ sıfır) `1011 x 101` işleminde satırların onlu katkılarını ($11 + 0 + 44 = 55$) toplayarak doğruladım.

Deney 4 - Bölüm kalan ve kesirli bölme algoritması

1. Bitleri soldan sağa işleyen bölme algoritmasını hazır bölme/kalan kullanmadan `ikili_bolucu.py` içinde yazdım. Sıfır bölen (`0` veya `000`) açık hata (`SifiraBolmeHatasi`) olarak reddedilir. Her adımda incelenen parça bölenle karşılaştırılır; büyük/eşitse bölüm biti `1` yazılıp fark alınır, küçükse `0` yazılıp parça korunur.

2-3. Üç zorunlu işlemin tam bölüm Q, kalan R, adım kayıtları ve kesirli sonuçları:

İşlem	Bölüm Q	Kalan R	$D = Q \times d + R$	$0 \le R < d$	Kesirli sonuç	Tekrar eden kalan
`10110 ÷ 100`	`101` (5)	`10` (2)	$22 = 5 \times 4 + 2$	$0 \le 2 < 4$	`= 101.1` (5.5)	Yok (1. adımda 0)
`1101 ÷ 10`	`110` (6)	`1` (1)	$13 = 6 \times 2 + 1$	$0 \le 1 < 2$	`= 110.1` (6.5)	Yok (1. adımda 0)
`111 ÷ 11`	`10` (2)	`1` (1)	$7 = 2 \times 3 + 1$	$0 \le 1 < 3$	$\approx$ `10.01010101`	`10, 1, 10, 1...`

- **Tam bölme adımları (`incelenen -> q_biti, kalan`):** `10110 ÷ 100`: `1->0,1` | `10->0,10` | `101->1,1` | `11->0,11` | `110->1,10`. `1101 ÷ 10`: `1->0,1` | `11->1,1` | `10->1,0` | `1->0,1`. `111 ÷ 11`: `1->0,1` | `11->1,0` | `1->0,1`.
- **İki koşulun gerekliliği:** Sadece $D = Q \times d + R$ kontrol edilirse $22 \div 4$ için $Q = 4, R = 6$ ($4 \times 4 + 6 = 22$) gibi bölmenin bitmediği yanlış sonuçlar geçer; sadece $0 \le R < d$ kontrol edilirse $Q = 0, R = 2$ gibi bölüneni vermeyen sonuçlar geçer. İkisi birlikte (Q, R) çiftini tekil yapar.
- **Durma koşulları (`111 ÷ 11`):** Tam bölme bölünenin 3 biti bitince durur ($Q =$ `10`$, R =$ `1`). Kesirli bölme ise kalan `0` olunca (`=`) veya kalan sıfırlanmadan 8 kesir basamağı dolunca ($\approx$) durur; `111 ÷ 11`'de kalanlar `1` ve `10` arasında sonsuz döngüye girdiği için 8. basamakta $\approx$ `10.01010101` olarak kesilmiştir.

4. Sınır testleri (Beklenen ve Gözlenen karşılaştırması) ve ara sıfır bitinin yanlış atılması örneği:

Sınır Senaryosu	İşlem	Beklenen Sonuç	Gözlenen Sonuç	Durum
Sıfır bölünen	`0000 ÷ 101`	$Q =$ `0`$, R =$ `0`, Kesirli `= 0`	$Q =$ `0`$, R =$ `0`, Kesirli `= 0`	Eşleşti
Bölünenden büyük bölen	`011 ÷ 110`	$Q =$ `0`$, R =$ `11`, Kesirli `= 0.1`	$Q =$ `0`$, R =$ `11`, Kesirli `= 0.1`	Eşleşti
Eşit operandlar	`101 ÷ 101`	$Q =$ `1`$, R =$ `0`, Kesirli `= 1`	$Q =$ `1`$, R =$ `0`, Kesirli `= 1`	Eşleşti
Ara sıfır bitli bölüm	`1010 ÷ 10`	$Q =$ `101`$, R =$ `0`, Kesirli `= 101`	$Q =$ `101`$, R =$ `0`, Kesirli `= 101`	Eşleşti
Sıfır bölen	`101 ÷ 000`	Hata (`SifiraBolmeHatasi`)	Hata (`SifiraBolmeHatasi`)	Eşleşti

Ara sıfır bitini yakalayan örnek ve YZ değerlendirmem (Deney 4): `1010 ÷ 10` ($10 \div 2 = 5$) işleminde 3. adımda parça `1 < 10` olduğu için bölüme `0` yazılmalıdır ($Q =$ `101` $= 5, R =$ `0`); bu sıfır biti yanlışlıkla atılırsa bölüm `11` (3) çıkar ve $3 \times 2 + 0 = 6 \neq 10$ kontrolüyle yakalanır. YZ'nin önerdiği kesir döngüsünü de `111 ÷ 11` işleminde kalanların `10` ve `1` diye tekrar etmesini elle takip ederek doğruladım.